



Facultad de Ingeniería  
Escuela de Ingeniería Informática

# **GUÍA TÉCNICA DE INSTALACIÓN, DESPLIEGUE E IMPLANTACIÓN DE APP ACOMPAÑAMIENTO UV**

Por

Sebastián Alejandro González Morales

Valparaíso, Septiembre 2026

# Índice

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                   | <b>1</b>  |
| <b>2. Requerimientos</b>                                 | <b>1</b>  |
| 2.1. Requerimientos mínimos del dispositivo Android      | 1         |
| 2.2. Requerimientos recomendados del dispositivo Android | 1         |
| 2.3. Requerimientos del entorno de desarrollo            | 2         |
| <b>3. Preparación del ambiente</b>                       | <b>2</b>  |
| 3.1. Herramientas y servicios                            | 2         |
| 3.2. Obtención del código fuente                         | 3         |
| <b>4. Estructura del proyecto</b>                        | <b>3</b>  |
| 4.1. Estructura general                                  | 3         |
| 4.2. Estructura del backend                              | 4         |
| 4.3. Estructura del frontend                             | 6         |
| <b>5. Ejecución en un entorno local</b>                  | <b>7</b>  |
| 5.1. Preparación del frontend                            | 7         |
| 5.2. Configuración de Android Studio                     | 7         |
| 5.3. Creación del emulador                               | 8         |
| 5.4. Generación del build de desarrollo                  | 8         |
| 5.5. Conexión con el backend local                       | 9         |
| 5.6. Preparación del backend                             | 10        |
| 5.7. Ejecución y verificación del backend                | 10        |
| <b>6. Construcción y despliegue del frontend</b>         | <b>11</b> |
| 6.1. Perfiles de EAS                                     | 11        |
| 6.2. Generación del archivo APK                          | 12        |
| 6.3. Generación del archivo AAB                          | 12        |
| 6.4. Configuración de la conexión en la nube             | 13        |
| <b>7. Despliegue del backend en Heroku</b>               | <b>14</b> |
| 7.1. Acceso al proyecto y autenticación                  | 15        |
| 7.2. Creación o asociación de la aplicación              | 15        |
| 7.3. Archivos requeridos por Heroku                      | 15        |
| 7.4. Variables de entorno                                | 16        |
| 7.5. Preparación y publicación                           | 16        |
| 7.6. Verificación del despliegue                         | 19        |
| <b>8. Instalación de la aplicación en Android</b>        | <b>22</b> |
| 8.1. Requisito previo                                    | 22        |
| 8.2. Descarga del APK                                    | 22        |
| 8.3. Permiso de instalación                              | 22        |
| <b>9. Criterios de continuidad</b>                       | <b>24</b> |

# 1. Introducción

Esta guía reúne y ordena la información de instalación, despliegue e implantación desarrollada para App Acompañamiento UV. Su propósito es permitir que otro estudiante o equipo pueda obtener el código, comprender su estructura, preparar un ambiente local, ejecutar sus componentes y reproducir el despliegue utilizado durante el proyecto.

El sistema se compone de dos módulos principales:

- **Frontend:** aplicación móvil desarrollada con React Native y Expo.
- **Backend:** API REST desarrollada con Node.js y Express, conectada mediante Mongoose a MongoDB Atlas.

La versión descrita genera una aplicación Android y mantiene el backend desplegado en Heroku. Las instrucciones reflejan el entorno utilizado durante el desarrollo y la validación técnica. Una implantación institucional definitiva requerirá revisar los accesos, la infraestructura, el soporte y el tratamiento de datos con las unidades y el equipo que continúen el desarrollo.

## 2. Requerimientos

Esta sección presenta las condiciones utilizadas como referencia para ejecutar la aplicación y preparar el ambiente de desarrollo. Se distinguen los requisitos del dispositivo Android y los del equipo empleado para mantener el frontend y el backend.

### 2.1. Requerimientos mínimos del dispositivo Android

- **Sistema operativo:** Android 8.0 o superior.
- **Memoria RAM:** al menos 2 GB.
- **Almacenamiento:** el APK utilizado durante el proyecto tiene un tamaño aproximado de 82 MB. Una vez instalada, la aplicación ocupa cerca de 116 MB y puede aumentar por los datos y la caché. Se recomienda disponer de al menos 200 MB libres.
- **Conexión a Internet:** red Wi-Fi o datos móviles estables.
- **Procesador:** capacidad suficiente para ejecutar una aplicación React Native; como referencia, un procesador equivalente o superior a un Qualcomm Snapdragon 450.

### 2.2. Requerimientos recomendados del dispositivo Android

- **Sistema operativo:** Android 10 o superior.
- **Memoria RAM:** 3 GB o más.
- **Almacenamiento:** 250 MB libres o más para permitir actualizaciones y crecimiento de la caché.

- **Conexión a Internet:** conexión estable para consultar y almacenar información en el backend.
- **Procesador:** un procesador de gama media o equivalente.

## 2.3. Requerimientos del entorno de desarrollo

Para trabajar con el proyecto se requiere un equipo que permita ejecutar Node.js, Android Studio y un emulador Android. Android Studio establece requisitos distintos según se utilice únicamente el editor o también el emulador; por esta razón, antes de instalarlo se deben revisar los requisitos vigentes en su sitio oficial.

El backend necesita acceso a MongoDB Atlas y a las variables de configuración del entorno. Para el despliegue utilizado en el proyecto también se requiere acceso autorizado a Heroku, mientras que la generación de builds móviles requiere una cuenta de Expo.

## 3. Preparación del ambiente

### 3.1. Herramientas y servicios

Antes de clonar el repositorio se deben instalar o preparar las herramientas de la Tabla 1. Los enlaces dirigen a la documentación o descarga oficial de cada producto.

Tabla 1: Herramientas utilizadas en el proyecto

| Herramienta        | Enlace y propósito                                                                                                                                                         |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Git y Git Bash     | <a href="https://git-scm.com/downloads">git-scm.com/downloads</a> . Permiten clonar y versionar el repositorio; Git Bash se incluye en la instalación de Git para Windows. |
| Visual Studio Code | <a href="https://code.visualstudio.com/Download">code.visualstudio.com/Download</a> . Editor utilizado para trabajar con el frontend y el backend.                         |
| Node.js y npm      | <a href="https://nodejs.org/en/download">nodejs.org/en/download</a> . Node.js ejecuta el proyecto y npm instala sus dependencias.                                          |
| MongoDB Atlas      | <a href="https://mongodb.com/docs/atlas">mongodb.com/docs/atlas</a> . Servicio de base de datos utilizado por el backend.                                                  |
| Android Studio     | <a href="https://developer.android.com/studio">developer.android.com/studio</a> . Incluye el SDK, las herramientas de Android y la administración de emuladores.           |
| Expo CLI           | <a href="https://docs.expo.dev/more/expo-cli">docs.expo.dev/more/expo-cli</a> . Se utiliza mediante <code>npx expo</code> para ejecutar la aplicación.                     |
| EAS CLI            | <a href="https://docs.expo.dev/build/setup">docs.expo.dev/build/setup</a> . Permite generar builds de desarrollo, APK y AAB.                                               |
| Heroku CLI         | <a href="https://devcenter.heroku.com/articles/heroku-cli">devcenter.heroku.com/articles/heroku-cli</a> . Administra el despliegue y los registros del backend.            |
| Postman            | <a href="https://learning.postman.com">learning.postman.com</a> . Permite realizar peticiones y comprobar los endpoints de la API.                                         |

Después de instalar Node.js, se puede instalar EAS CLI con:

```
npm install -g eas-cli
```

Expo CLI forma parte del paquete Expo del proyecto y se ejecuta mediante `npx expo`; no es necesario instalar globalmente el paquete antiguo `expo-cli`.

### 3.2. Obtención del código fuente

El repositorio público del proyecto se encuentra en:

<https://github.com/SebastianGonzalezMorales/AppAcompanhamentoUV>

Para clonarlo, se abre Git Bash o una terminal en la carpeta donde se desea guardar el proyecto y se ejecuta:

```
git clone https://github.com/SebastianGonzalezMorales/  
AppAcompanhamentoUV.git  
cd AppAcompanhamentoUV
```

Al ser un repositorio público, la lectura y clonación no requieren credenciales del proyecto. Esta disponibilidad no implica una licencia de uso; al cierre del proyecto, dicha licencia no se encontraba definida. La Figura 1 muestra el comando utilizado.

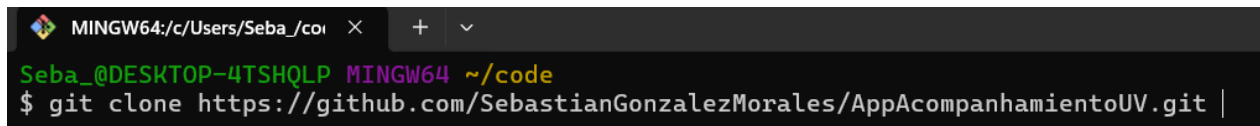


Figura 1: Ejecución del comando `git clone` en la terminal.

## 4. Estructura del proyecto

La aplicación mantiene el frontend y el backend en un mismo repositorio. La estructura permite trabajar cada componente por separado y conservar en la raíz los archivos utilizados por Heroku.

### 4.1. Estructura general



Figura 2: Estructura general de carpetas del proyecto.

La carpeta raíz contiene:

- **Backend:** código del servidor y de la API REST.
- **Frontend:** aplicación móvil desarrollada con React Native y Expo.
- **package.json:** archivo ubicado en la raíz para apoyar la detección del proyecto durante el despliegue.
- **Procfile:** instrucción utilizada por Heroku para iniciar el backend que se encuentra en una subcarpeta.

## 4.2. Estructura del backend

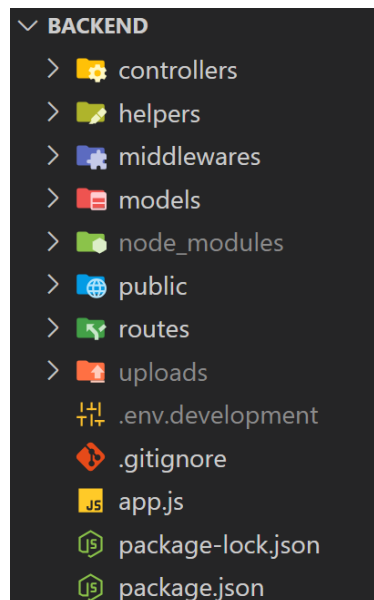


Figura 3: Estructura de la carpeta Backend.

Los elementos principales de la carpeta Backend son:

- **controllers:** controladores que reciben las peticiones HTTP, ejecutan la lógica de cada operación y coordinan el acceso a los modelos.
- **helpers:** funciones auxiliares reutilizadas por los controladores u otros módulos.
- **middlewares:** funciones que se ejecutan antes de los controladores, por ejemplo para validar tokens, procesar archivos o manejar errores.
- **models:** esquemas de Mongoose que definen la estructura y validación de las colecciones almacenadas en MongoDB.
- **node\_modules:** dependencias instaladas por npm. Esta carpeta se genera localmente y no debe versionarse.

- **public:** recursos públicos o estáticos empleados por el backend.
- **routes:** rutas de la API y asociación de cada endpoint con su controlador.
- **uploads:** imágenes almacenadas por la API, por ejemplo fotografías utilizadas por los recursos institucionales.
- **.env.development:** variables de entorno del desarrollo local. Debe permanecer fuera del repositorio público.
- **app.js:** punto de entrada del backend; inicializa Express, los middlewares, las rutas y la conexión con la base de datos.
- **package.json:** scripts y dependencias del backend.
- **package-lock.json:** versiones exactas de las dependencias instaladas.
- **.gitignore:** archivos y carpetas que Git debe excluir, especialmente dependencias y configuración sensible.

### 4.3. Estructura del frontend

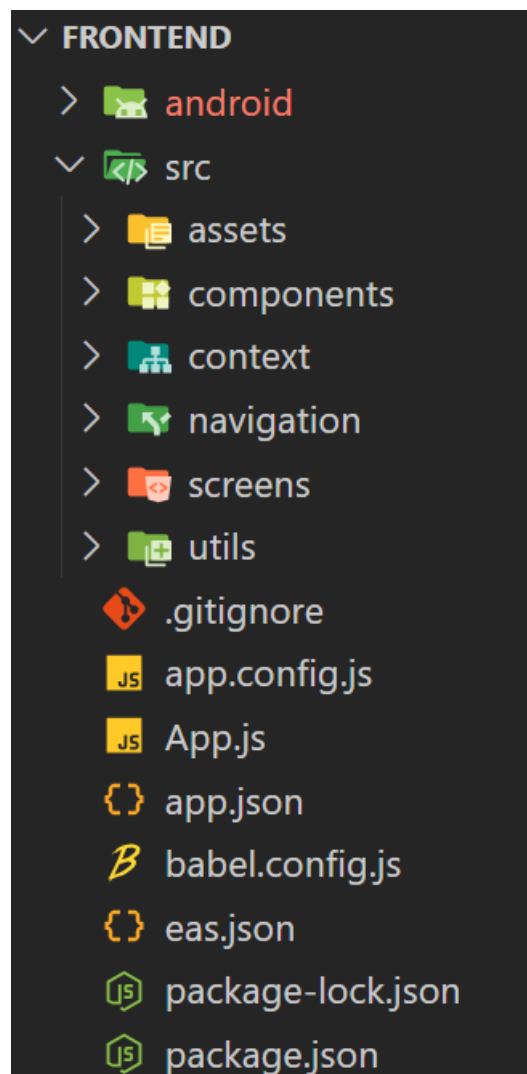


Figura 4: Estructura de la carpeta Frontend.

Los elementos principales de la carpeta `Frontend` son:

- **android**: archivos nativos generados para preparar y construir la aplicación Android.
- **src**: código fuente de la aplicación, organizado en:
  - **assets**: imágenes, fuentes, estilos y otros recursos estáticos.
  - **components**: componentes reutilizables de la interfaz.
  - **context**: autenticación global, token y verificación de sesión mediante React Context y AsyncStorage.
  - **navigation**: navegadores y rutas principales de la aplicación.

- **screens**: pantallas organizadas según las funcionalidades de la aplicación.
- **utils**: funciones auxiliares, constantes y utilidades compartidas.
- **app.config.js**: expone al proyecto las variables de entorno utilizadas por Expo.
- **App.js**: punto de entrada del frontend; configura el contexto y la navegación.
- **app.json**: configuración general de Expo, como nombre, iconos y versión.
- **babel.config.js**: configuración de Babel utilizada para transformar el código JavaScript.
- **eas.json**: perfiles de construcción utilizados por EAS.
- **package.json**: scripts y dependencias del frontend.
- **package-lock.json**: versiones exactas de las dependencias instaladas.
- **.gitignore**: exclusiones del frontend, como dependencias, builds y archivos locales.

## 5. Ejecución en un entorno local

### 5.1. Preparación del frontend

Desde la raíz del repositorio se accede al frontend y se instalan sus dependencias:

```
cd AppAcompañamientoUV/Frontend/
npm install
```

El tiempo de instalación depende de la conexión a Internet. Cuando termina, se debe preparar Android Studio y el dispositivo o emulador donde se ejecutará la aplicación.

### 5.2. Configuración de Android Studio

Al abrir Android Studio por primera vez, el asistente de configuración instala los componentes necesarios:

- SDK de Android;
- herramientas de línea de comandos, como `adb` y `emulator`;
- administrador de dispositivos virtuales Android (AVD);
- componentes básicos para construir y ejecutar aplicaciones Android.

En Windows, las herramientas deben quedar disponibles para las terminales externas. La Tabla 2 resume las variables utilizadas.

Tabla 2: Variables del entorno Android en Windows

| Variable     | Contenido                                       | Función                                                  |
|--------------|-------------------------------------------------|----------------------------------------------------------|
| ANDROID_HOME | Ruta local donde Android Studio instaló el SDK. | Permite que Expo y otras herramientas encuentren el SDK. |
| Path         | Carpetas emulator y platform-tools del SDK.     | Permite ejecutar adb y emulator desde la terminal.       |

Para configurar estas variables:

1. Abrir *Configuración avanzada del sistema* y seleccionar *Variables de entorno*.
2. Crear ANDROID\_HOME con la ruta donde Android Studio instaló el SDK.
3. Editar Path y agregar:

```
%ANDROID_HOME%\emulator
%ANDROID_HOME%\platform-tools
```

4. Guardar los cambios y reiniciar Android Studio. Si la terminal no reconoce las variables, se debe cerrar y abrir nuevamente la sesión.

### 5.3. Creación del emulador

Para crear un dispositivo virtual en Android Studio:

1. Abrir *Device Manager* desde la pantalla inicial o desde el menú de herramientas.
2. Seleccionar *Create Device*, elegir la categoría *Phone* y un modelo compatible.
3. Descargar y seleccionar una imagen de Android compatible con el proyecto.
4. Asignar un nombre al emulador, finalizar su creación e iniciarlo desde *Device Manager*.

La primera ejecución puede tardar algunos minutos. Cuando aparece la pantalla de inicio de Android, el dispositivo está listo para recibir el build de desarrollo.

### 5.4. Generación del build de desarrollo

Desde la carpeta Frontend, se ejecuta:

```
eas login
eas build --profile development --platform android
```

EAS solicita una cuenta de Expo autorizada y entrega un enlace para consultar el avance. Al finalizar, puede instalar el build en el emulador seleccionado. La configuración de EAS y las cuentas de los servicios deben ser entregadas de forma segura a la persona que continúe el desarrollo; no se incluyen credenciales en esta guía.

## 5.5. Conexión con el backend local

La aplicación móvil y el backend se ejecutan como procesos separados. Desde el emulador o un dispositivo físico, `localhost` apunta al propio dispositivo y no al computador donde está el servidor. Por esta razón, se utiliza la dirección IPv4 del equipo de desarrollo.

En Windows, la dirección se obtiene con:

```
ipconfig
```

Luego se identifican la conexión activa y su dirección IPv4. Con el backend disponible en el puerto correspondiente, el frontend se inicia desde Git Bash reemplazando la dirección de ejemplo:

```
API_URL=http://192.168.1.7:3001/api/v1
BASE_URL=http://192.168.1.7:3001
npx expo start --dev-client
```

Cuando Expo inicia, se mantiene abierto el emulador y se presiona la tecla `a` para abrir la aplicación Android. La Figura 5 muestra el servidor de Expo y el emulador funcionando en conjunto.

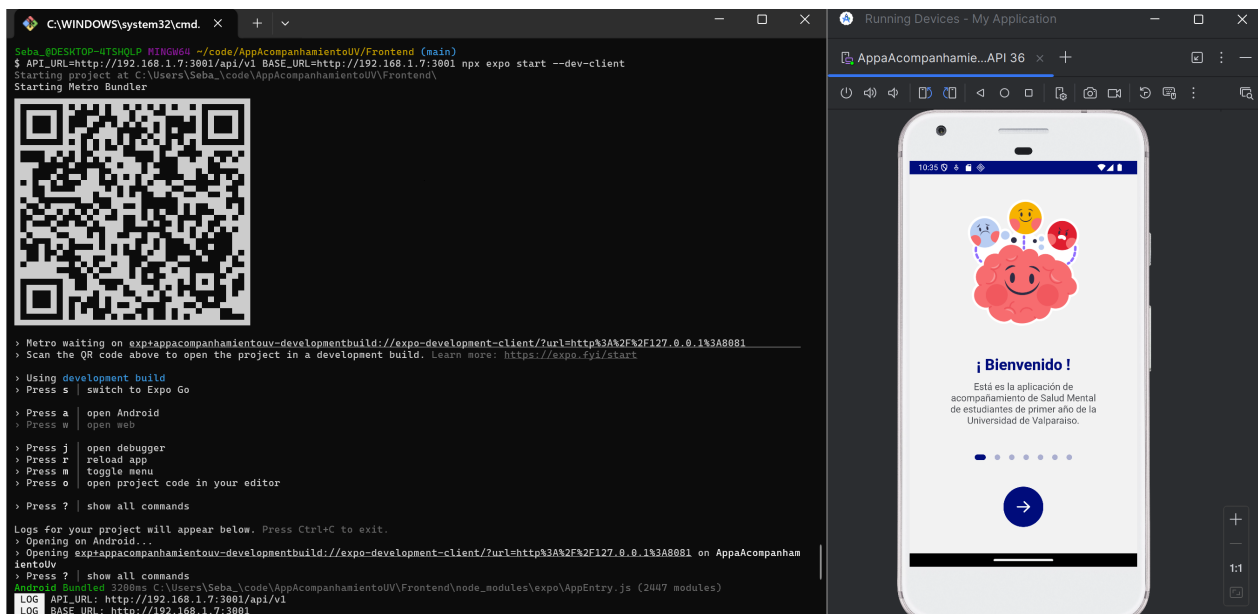


Figura 5: Aplicación en ejecución mediante Expo y un emulador Android.

Con la aplicación en ejecución, el proyecto puede abrirse en Visual Studio Code:

```
cd AppAcompañamientoUV/Frontend/
code .
```

Los cambios guardados se reflejan mediante el servidor de desarrollo mientras el emulador y Expo continúen activos.

## 5.6. Preparación del backend

En una nueva terminal se accede al backend y se instalan sus dependencias:

```
cd AppAcompañamientoUV/Backend/  
npm install
```

El backend carga su configuración local desde `.env.development`. Este archivo debe contener los valores autorizados para el entorno, entre ellos las direcciones del servidor, la conexión con MongoDB Atlas y las claves utilizadas por la aplicación. Los nombres pueden incluir:

```
API_URL=<ruta_base_de_la_api>  
BASE_URL_DEV=<direccion_local>  
BASE_URL_PROD=<direccion_remota>  
CONNECTION_STRING=<conexion_mongodb_atlas>  
SECRET=<secreto_de_autenticacion>  
EMAIL_USER=<cuenta_de_correo>  
EMAIL_PASS=<credencial_de_correo>  
ENCRYPTION_KEY=<clave_de_cifrado>  
SIGNING_KEY=<clave_de_firma>
```

Los valores se obtienen de la configuración vigente del entorno y se transmiten por un canal seguro. El archivo no debe incluirse en Git ni copiarse a documentación pública.

## 5.7. Ejecución y verificación del backend

El servidor local se inicia con:

```
npm run dev
```

Este comando utiliza Nodemon para reiniciar el servidor cuando se detectan cambios. La salida esperada confirma la carga del entorno, el puerto y la conexión con la base de datos, como se observa en la Figura 6.

```
C:\WINDOWS\system32\cmd. X + v
Seba_@DESKTOP-4TSHQLP MINGW64 ~/codigo/development-AppAcompañamientoUv/Backend (main)
$ npm run dev

> my-app@1.0.0 dev
> cross-env NODE_ENV=development nodemon app.js

[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node app.js`

=====
Archivo .env cargado para el entorno: development ✂.
=====

🔑 Variables de entorno cargadas:
=====

API_URL:                /a...
BASE_URL_DEV:           ht...
JWT_SECRET:             06...
CONNECTION_STRING:      mo...
EMAIL_USER:             ap...
EMAIL_PASS:             ap...
ENCRYPTION_KEY:         8a...
SIGNING_KEY:            b5...

SERVIDOR CORRIENDO EN: http://localhost:3001 ●.

Conexión a la base de datos lista ✔.
```

Figura 6: Ejecución del backend en el entorno local; los valores sensibles aparecen abreviados.

El servidor utilizado durante el desarrollo queda disponible en <http://localhost:3001>. Postman permite enviar peticiones a <http://localhost:3001/api/v1/> seguida de la ruta que se desea comprobar. Antes de conectar el frontend, se recomienda verificar al menos un endpoint de lectura y uno de escritura.

## 6. Construcción y despliegue del frontend

### 6.1. Perfiles de EAS

El archivo `eas.json` mantiene tres perfiles:

- **development:** build con herramientas de desarrollo para el emulador o dispositivo de pruebas.
- **preview:** genera un APK instalable directamente en Android.
- **production:** genera un Android App Bundle (AAB) orientado a una eventual publicación en Google Play.

```

eas.json > ...
1  {
2    "cli": {
3      "version": ">= 12.5.1",
4      "appVersionSource": "remote"
5    },
6    "build": {
7      "development": {
8        "developmentClient": true,
9        "distribution": "internal",
10       "android": {
11         "buildType": "apk"
12       }
13     },
14     "preview": {
15       "distribution": "internal",
16       "android": {
17         "buildType": "apk"
18       }
19     },
20     "production": {
21       "autoIncrement": true,
22       "distribution": "store",
23       "android": {
24         "buildType": "app-bundle"
25       }
26     },
27     "submit": {
28       "production": {}
29     }
30   }
31 }
32 }
33 }
34 }

```

Figura 7: Perfiles de construcción configurados en el archivo `eas.json`.

## 6.2. Generación del archivo APK

Desde la carpeta `Frontend`, se genera el APK con:

```
eas build --profile preview --platform android
```

Al terminar, EAS entrega un enlace para descargar el archivo. El APK permite instalar la aplicación directamente en dispositivos Android y fue el formato utilizado durante las pruebas del proyecto.

## 6.3. Generación del archivo AAB

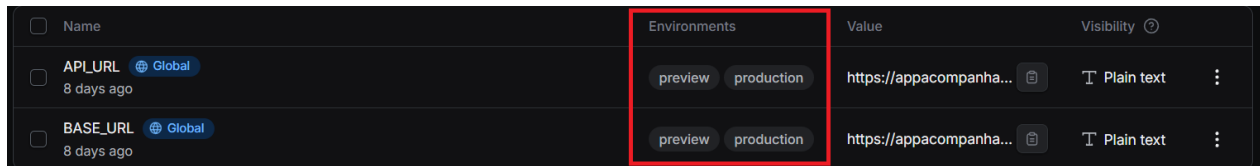
Para generar un paquete orientado a Google Play:

```
eas build --profile production --platform android
```

El comando produce un archivo AAB. Su generación no implica que la aplicación esté publicada; la creación de la ficha, el cumplimiento de las políticas y la distribución en Google Play no forman parte del despliegue actual. La documentación oficial está disponible en [Publicar una aplicación en Google Play](#).

## 6.4. Configuración de la conexión en la nube

Para que el APK se comunice con Heroku, las variables `API_URL` y `BASE_URL` deben apuntar al backend remoto en los perfiles `preview` y `production`. La Figura 8 muestra la configuración utilizada en la plataforma de Expo.



| Name                   | Environments       | Value                   | Visibility |
|------------------------|--------------------|-------------------------|------------|
| APLURL<br>8 days ago   | preview production | https://appacompanha... | Plain text |
| BASE_URL<br>8 days ago | preview production | https://appacompanha... | Plain text |

Figura 8: Variables de entorno asociadas a los perfiles de Expo.

El archivo `app.config.js` incorpora las variables proporcionadas por Expo dentro del objeto `extra`. De esta forma, la misma base de código puede utilizar valores distintos según el entorno.

```
app.config.js > ...
1 export default ({ config }) => {
2   return {
3     ...config,
4     extra: {
5       ...config.extra,
6       API_URL: process.env.API_URL,
7       BASE_URL: process.env.BASE_URL,
8     },
9   };
10 };
```

Figura 9: Acceso a las variables de entorno desde `app.config.js`.

Posteriormente, `apiHelpers.js` obtiene `API_URL` mediante `expo-constants` y la utiliza como base para construir las solicitudes al backend.

```

src > utils >  apiHelpers.js > ...
1  import axios from 'axios';
2  import AsyncStorage from '@react-native-async-storage/async-storage';
3  import Constants from 'expo-constants';
4
5  // Se extrae API_URL desde la configuración establecida en app.config.js
6  const { API_URL } = Constants.expoConfig?.extra || {};
7
8  export const fetchWithToken = async (endpoint, method = 'GET', data = null, params = null) => {
9    try {
10      // Obtener el token desde AsyncStorage
11      const token = await AsyncStorage.getItem('token');
12
13      // Configuración de la solicitud con la URL del backend
14      const config = {
15        method,
16        url: `${API_URL}${endpoint}`,
17        headers: {
18          Authorization: `Bearer ${token}`,
19        },
20      };
21      // Configuración adicional...
22
23      // Realizar la solicitud al backend
24      const response = await axios(config);
25      return response.data;
26    } catch (error) {
27      // Manejo de errores...
28    }
29  };

```

Figura 10: Uso de API\_URL en el archivo apiHelpers.js.

Después de instalar el build, se debe verificar la conexión mediante acciones representativas:

- iniciar sesión con credenciales válidas y comprobar la respuesta ante datos incorrectos;
- consultar la información almacenada;
- registrar un estado de ánimo y comprobar que quede disponible en el historial;
- realizar el PHQ-9 y confirmar que su resultado se conserve.

## 7. Despliegue del backend en Heroku

Esta sección conserva el procedimiento aplicado durante el proyecto. Las direcciones y nombres mostrados corresponden al entorno documentado; si se crea una aplicación nueva, deben reemplazarse por los datos del nuevo entorno.

## 7.1. Acceso al proyecto y autenticación

Desde la carpeta del backend:

```
cd AppAcompañamientoUV/Backend/  
heroku login
```

El segundo comando abre el navegador para completar la autenticación. No se deben compartir tokens ni copiar credenciales en el repositorio.

## 7.2. Creación o asociación de la aplicación

Para crear un entorno nuevo se utiliza un nombre disponible:

```
heroku create <nombre-de-la-aplicacion>
```

Si la aplicación ya existe, el repositorio local puede asociarse con:

```
heroku git:remote -a <nombre-de-la-aplicacion>
```

La Figura 11 corresponde a la creación del entorno utilizado durante el proyecto.

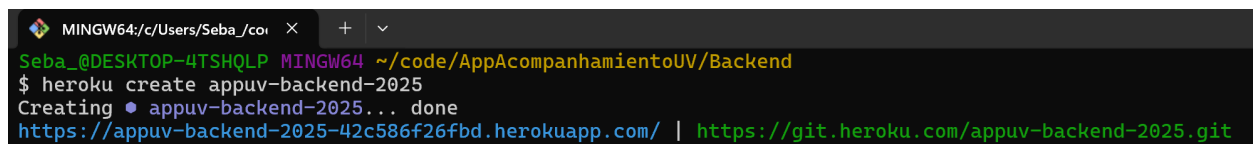


Figura 11: Creación de la aplicación del backend en Heroku.

## 7.3. Archivos requeridos por Heroku

El archivo `Procfile`, ubicado en la raíz del repositorio y sin extensión, indica cómo iniciar el backend:

```
web: cd Backend && npm install && npm start
```

En el archivo `Backend/package.json` se comprueba la existencia de los scripts:

```
"scripts": {  
  "start": "cross-env NODE_ENV=production node app.js",  
  "dev": "cross-env NODE_ENV=development nodemon app.js"  
}
```

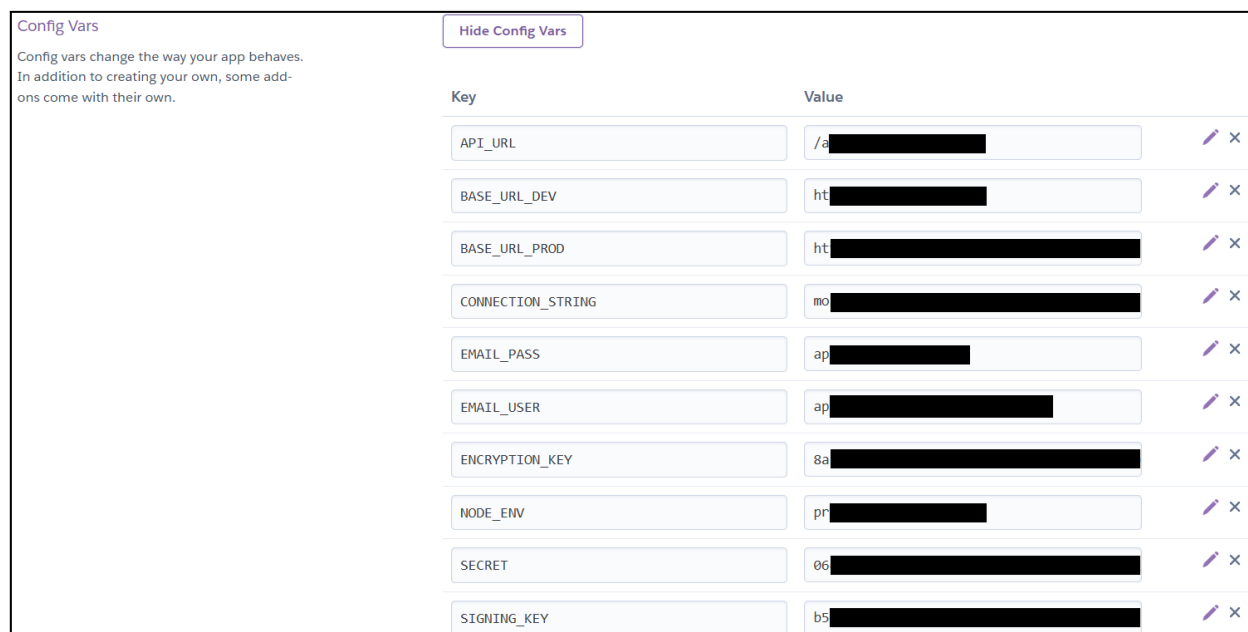
El script `start` es utilizado por Heroku para iniciar `app.js` en producción. El script `dev` se utiliza localmente con Nodemon.

## 7.4. Variables de entorno

Las variables se configuran desde el panel de Heroku:

1. Ingresar a <https://dashboard.heroku.com>.
2. Seleccionar la aplicación.
3. Abrir *Settings* y localizar *Config Vars*.
4. Agregar cada nombre y su valor correspondiente.

La configuración utilizada incluye las direcciones del backend, la cadena de conexión con MongoDB Atlas y las claves necesarias para la autenticación, cifrado, firma y envío de correos. La Figura 12 conserva la evidencia del entorno, con los valores sensibles ocultos.



| Key               | Value         |
|-------------------|---------------|
| API_URL           | /a [REDACTED] |
| BASE_URL_DEV      | ht [REDACTED] |
| BASE_URL_PROD     | ht [REDACTED] |
| CONNECTION_STRING | mo [REDACTED] |
| EMAIL_PASS        | ap [REDACTED] |
| EMAIL_USER        | ap [REDACTED] |
| ENCRYPTION_KEY    | 8a [REDACTED] |
| NODE_ENV          | pr [REDACTED] |
| SECRET            | 06 [REDACTED] |
| SIGNING_KEY       | bs [REDACTED] |

Figura 12: Variables de entorno configuradas en Heroku; los valores sensibles están ocultos.

## 7.5. Preparación y publicación

Antes de desplegar, se revisa el estado del repositorio:

```
git status
git remote -v
```

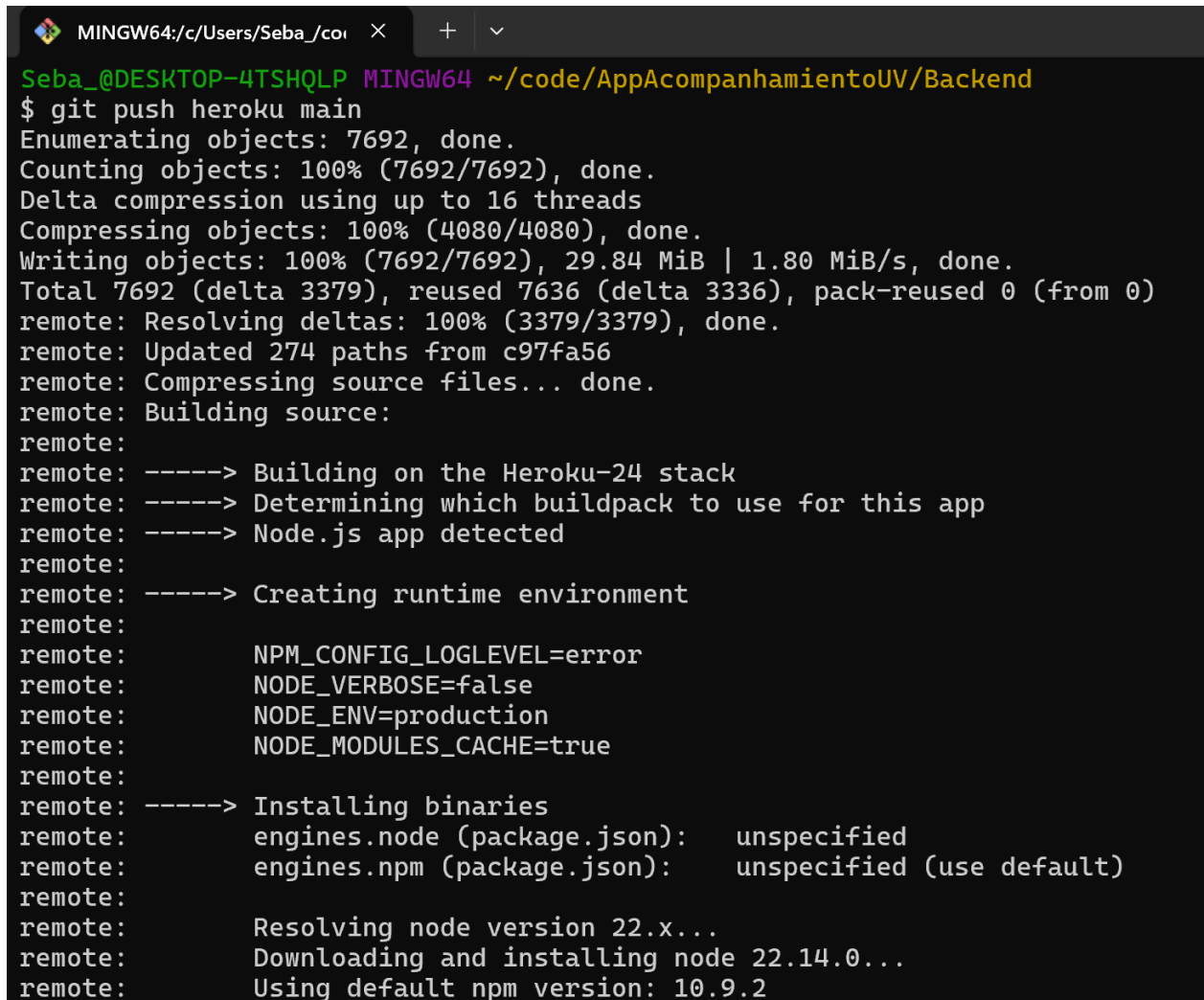
Si existen cambios que deben formar parte de la versión:

```
git add .
git commit -m "Configuracion para despliegue en Heroku"
```

Con el remoto heroku asociado, el despliegue se inicia con:

```
git push heroku main
```

Heroku recibe el código, detecta Node.js, instala las dependencias y ejecuta el proceso definido en el Procfile. Las Figuras 13, 14 y 15 muestran las etapas registradas durante el proyecto.



```
MINGW64:/c/Users/Seba_/cor X + v
Seba_@DESKTOP-4TSHQLP MINGW64 ~/code/AppAcompanhamentoUV/Backend
$ git push heroku main
Enumerating objects: 7692, done.
Counting objects: 100% (7692/7692), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4080/4080), done.
Writing objects: 100% (7692/7692), 29.84 MiB | 1.80 MiB/s, done.
Total 7692 (delta 3379), reused 7636 (delta 3336), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3379/3379), done.
remote: Updated 274 paths from c97fa56
remote: Compressing source files... done.
remote: Building source:
remote:
remote: -----> Building on the Heroku-24 stack
remote: -----> Determining which buildpack to use for this app
remote: -----> Node.js app detected
remote:
remote: -----> Creating runtime environment
remote:
remote:       NPM_CONFIG_LOGLEVEL=error
remote:       NODE_VERBOSE=false
remote:       NODE_ENV=production
remote:       NODE_MODULES_CACHE=true
remote:
remote: -----> Installing binaries
remote:       engines.node (package.json):  unspecified
remote:       engines.npm (package.json):   unspecified (use default)
remote:
remote:       Resolving node version 22.x...
remote:       Downloading and installing node 22.14.0...
remote:       Using default npm version: 10.9.2
```

Figura 13: Inicio del despliegue y detección del entorno Node.js.

```
MINGW64:/c/Users/Seba_/coi X + v
remote: -----> Installing dependencies
remote:       Installing node modules
remote:
remote:       added 510 packages, and audited 511 packages in 5s
remote:
remote:       33 packages are looking for funding
remote:       run `npm fund` for details
remote:
remote:       3 vulnerabilities (2 moderate, 1 high)
remote:
remote:       To address all issues, run:
remote:         npm audit fix
remote:
remote:       Run `npm audit` for details.
remote:       npm notice
remote:       npm notice New major version of npm available! 10.9.2 -> 11.3.0
remote:       npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.3.0
remote:       npm notice To update run: npm install -g npm@11.3.0
remote:       npm notice
remote: -----> Build
remote:
remote: -----> Caching build
remote:       - npm cache
remote: -----> Pruning devDependencies
remote:
remote:       up to date, audited 511 packages in 972ms
remote:
remote:       33 packages are looking for funding
remote:       run `npm fund` for details
```

Figura 14: Instalación de dependencias y construcción de la aplicación.

```
MINGW64:/c/Users/Seba/coi X + v
remote:      3 vulnerabilities (2 moderate, 1 high)
remote:
remote:      To address all issues, run:
remote:        npm audit fix
remote:
remote:      Run `npm audit` for details.
remote:      npm notice
remote:      npm notice New major version of npm available! 10.9.2 -> 11.3.0
remote:      npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.3.0
remote:      npm notice To update run: npm install -g npm@11.3.0
remote:      npm notice
remote:
remote: -----> Build succeeded!
remote: -----> Discovering process types
remote:      Procfile declares types -> web
remote:
remote: -----> Compressing...
remote:      Done: 147.4M
remote: -----> Launching...
remote:      Released v13
remote:      https://appuv-backend-2025-42c586f26fbd.herokuapp.com/ deployed to Heroku
remote:
remote: Verifying deploy... done.
To https://git.heroku.com/appuv-backend-2025.git
* [new branch]      main -> main
```

Figura 15: Finalización correcta del despliegue en Heroku.

## 7.6. Verificación del despliegue

Los registros en tiempo real se consultan mediante:

```
heroku logs --tail
```

En ellos se comprueba el inicio del proceso, la carga del entorno, la conexión con la base de datos y la recepción de solicitudes. Las siguientes capturas conservan la evidencia del despliegue realizado; cualquier valor sensible visible se encuentra abreviado.

```
MINGW64:/c/Users/Seba/_coi X + v
$ heroku logs --tail
2025-04-19T21:12:51.988757+00:00 app[api]: Initial release by user sgonzalez9045@gmail.com
2025-04-19T21:12:51.988757+00:00 app[api]: Release v1 created by user sgonzalez9045@gmail.com
2025-04-19T21:12:52.177579+00:00 app[api]: Release v2 created by user sgonzalez9045@gmail.com
2025-04-19T21:12:52.177579+00:00 app[api]: Enable Logplex by user sgonzalez9045@gmail.com
2025-04-20T02:11:42.777599+00:00 app[api]: Release v3 created by user sgonzalez9045@gmail.com
2025-04-20T02:11:42.777599+00:00 app[api]: Set API_URL config vars by user sgonzalez9045@gmail.com
2025-04-20T02:11:49.813888+00:00 app[api]: Set BASE_URL_DEV config vars by user sgonzalez9045@gmail.com
2025-04-20T02:11:49.813888+00:00 app[api]: Release v4 created by user sgonzalez9045@gmail.com
2025-04-20T02:12:01.147548+00:00 app[api]: Set BASE_URL_PROD config vars by user sgonzalez9045@gmail.com
2025-04-20T02:12:01.147548+00:00 app[api]: Release v5 created by user sgonzalez9045@gmail.com
2025-04-20T02:12:09.859234+00:00 app[api]: Release v6 created by user sgonzalez9045@gmail.com
2025-04-20T02:12:09.859234+00:00 app[api]: Set CONNECTION_STRING config vars by user sgonzalez9045@gmail.com
2025-04-20T02:12:14.916546+00:00 app[api]: Release v7 created by user sgonzalez9045@gmail.com
2025-04-20T02:12:14.916546+00:00 app[api]: Set EMAIL_PASS config vars by user sgonzalez9045@gmail.com
2025-04-20T02:12:23.604408+00:00 app[api]: Set EMAIL_USER config vars by user sgonzalez9045@gmail.com
2025-04-20T02:12:23.604408+00:00 app[api]: Release v8 created by user sgonzalez9045@gmail.com
2025-04-20T02:12:29.864466+00:00 app[api]: Release v9 created by user sgonzalez9045@gmail.com
2025-04-20T02:12:29.864466+00:00 app[api]: Set ENCRYPTION_KEY config vars by user sgonzalez9045@gmail.com
2025-04-20T02:12:36.196477+00:00 app[api]: Release v10 created by user sgonzalez9045@gmail.com
2025-04-20T02:12:36.196477+00:00 app[api]: Set NODE_ENV config vars by user sgonzalez9045@gmail.com
2025-04-20T02:12:46.000225+00:00 app[api]: Set SECRET config vars by user sgonzalez9045@gmail.com
2025-04-20T02:12:46.000225+00:00 app[api]: Release v11 created by user sgonzalez9045@gmail.com
2025-04-20T02:12:51.642518+00:00 app[api]: Set SIGNING_KEY config vars by user sgonzalez9045@gmail.com
2025-04-20T02:12:51.642518+00:00 app[api]: Release v12 created by user sgonzalez9045@gmail.com
2025-04-20T04:04:06.000000+00:00 app[api]: Build started by user sgonzalez9045@gmail.com
2025-04-20T04:04:33.182330+00:00 app[api]: Release v13 created by user sgonzalez9045@gmail.com
2025-04-20T04:04:33.182330+00:00 app[api]: Deploy 7d1141a8 by user sgonzalez9045@gmail.com
2025-04-20T04:04:33.198709+00:00 app[api]: Scaled to web@1:Eco by user sgonzalez9045@gmail.com
2025-04-20T04:04:37.000000+00:00 app[api]: Build succeeded
2025-04-20T04:04:38.837010+00:00 heroku[web.1]: Starting process with command 'cd Backend && npm install && npm start'
2025-04-20T04:04:39.757859+00:00 app[web.1]: npm warn EBADENGINE Unsupported engine {
2025-04-20T04:04:39.757859+00:00 app[web.1]:   package: 'my-app@1.0.0',
2025-04-20T04:04:39.757859+00:00 app[web.1]:   required: { node: '22.14.0', npm: '11.2.0' },
2025-04-20T04:04:39.757859+00:00 app[web.1]:   current: { node: 'v22.14.0', npm: '10.9.2' }
2025-04-20T04:04:39.757900+00:00 app[web.1]: npm warn EBADENGINE }
2025-04-20T04:04:41.129127+00:00 app[web.1]:
2025-04-20T04:04:41.129140+00:00 app[web.1]: added 176 packages, and audited 177 packages in 2s
2025-04-20T04:04:41.129192+00:00 app[web.1]:
2025-04-20T04:04:41.129210+00:00 app[web.1]: 24 packages are looking for funding
2025-04-20T04:04:41.129225+00:00 app[web.1]: run 'npm fund' for details
2025-04-20T04:04:41.131193+00:00 app[web.1]:
2025-04-20T04:04:41.131194+00:00 app[web.1]: 1 high severity vulnerability
2025-04-20T04:04:41.131194+00:00 app[web.1]:
2025-04-20T04:04:41.131195+00:00 app[web.1]: To address all issues (including breaking changes), run:
2025-04-20T04:04:41.131196+00:00 app[web.1]: npm audit fix --force
2025-04-20T04:04:41.131196+00:00 app[web.1]:
2025-04-20T04:04:41.131196+00:00 app[web.1]: Run 'npm audit' for details.
2025-04-20T04:04:41.131996+00:00 app[web.1]: npm notice
2025-04-20T04:04:41.131997+00:00 app[web.1]: npm notice New major version of npm available! 10.9.2 -> 11.3.0
2025-04-20T04:04:41.132002+00:00 app[web.1]: npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.3.0
2025-04-20T04:04:41.132003+00:00 app[web.1]: npm notice To update run: npm install -g npm@11.3.0
2025-04-20T04:04:41.132003+00:00 app[web.1]: npm notice
2025-04-20T04:04:41.250329+00:00 app[web.1]:
2025-04-20T04:04:41.250331+00:00 app[web.1]: > my-app@1.0.0 start
2025-04-20T04:04:41.250331+00:00 app[web.1]: > cross-env NODE_ENV=production node app.js
```

Figura 16: Registro de eventos y configuración del despliegue en Heroku.



## 8. Instalación de la aplicación en Android

### 8.1. Requisito previo

El dispositivo debe contar con Android 8.0 o superior, conexión a Internet y al menos 200 MB de espacio libre. Para permitir actualizaciones y datos en caché, se recomiendan 250 MB o más.

### 8.2. Descarga del APK

La versión utilizada durante el proyecto puede obtenerse mediante el siguiente enlace:

[Descargar App Acompañamiento UV](#)

El archivo debe descargarse únicamente desde un enlace controlado por el proyecto. Antes de instalarlo, se recomienda comprobar que la versión corresponda a la entrega esperada.

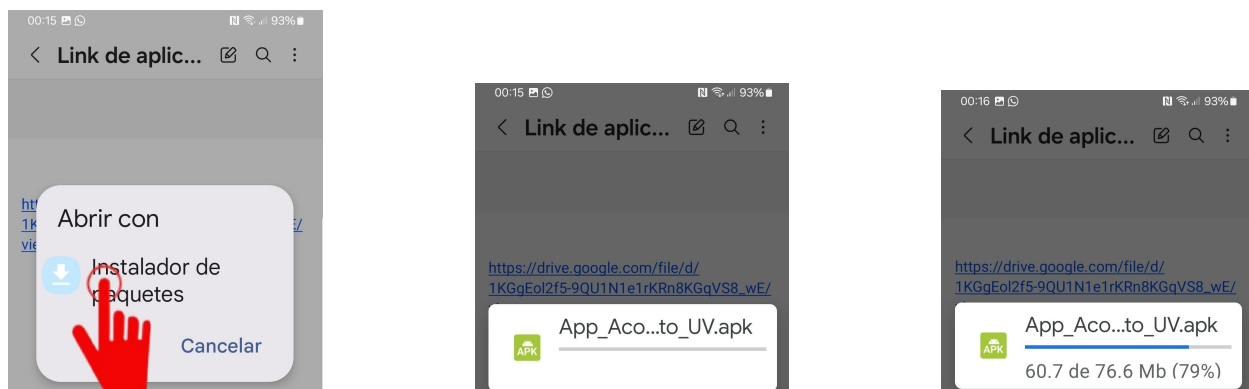


Figura 19: Apertura del instalador y progreso de descarga del archivo APK.

### 8.3. Permiso de instalación

Si Android muestra un aviso de seguridad, se abre *Configuración* y se autoriza temporalmente la instalación desde la aplicación utilizada para descargar el archivo. Luego se vuelve al instalador y se selecciona *Instalar*.

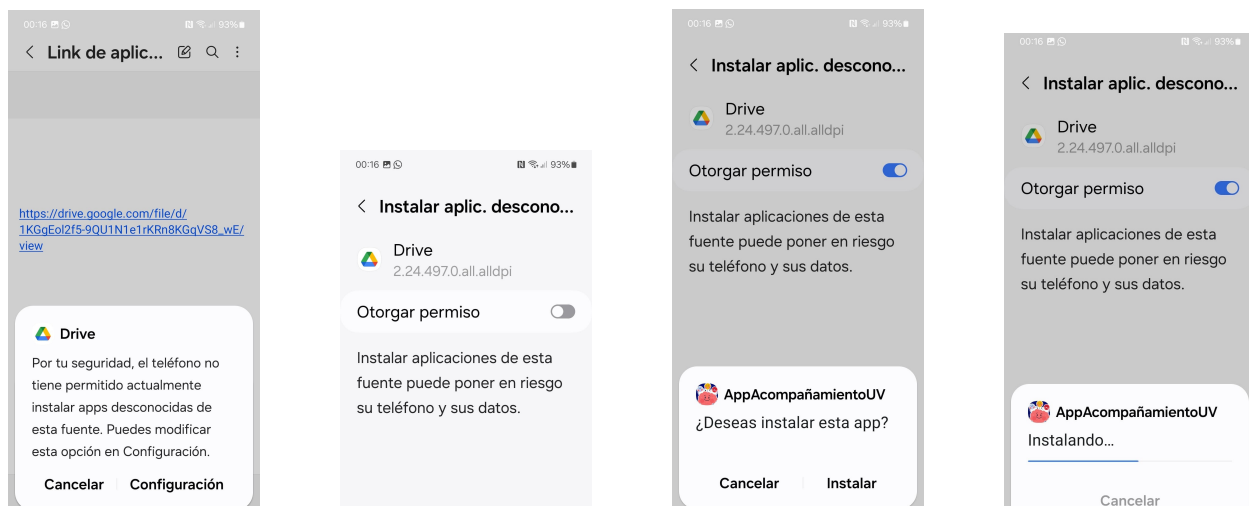


Figura 20: Autorización e instalación del APK en Android.

Cuando el proceso termina, se puede seleccionar *Abrir* para iniciar la aplicación o *Listo* para hacerlo posteriormente.

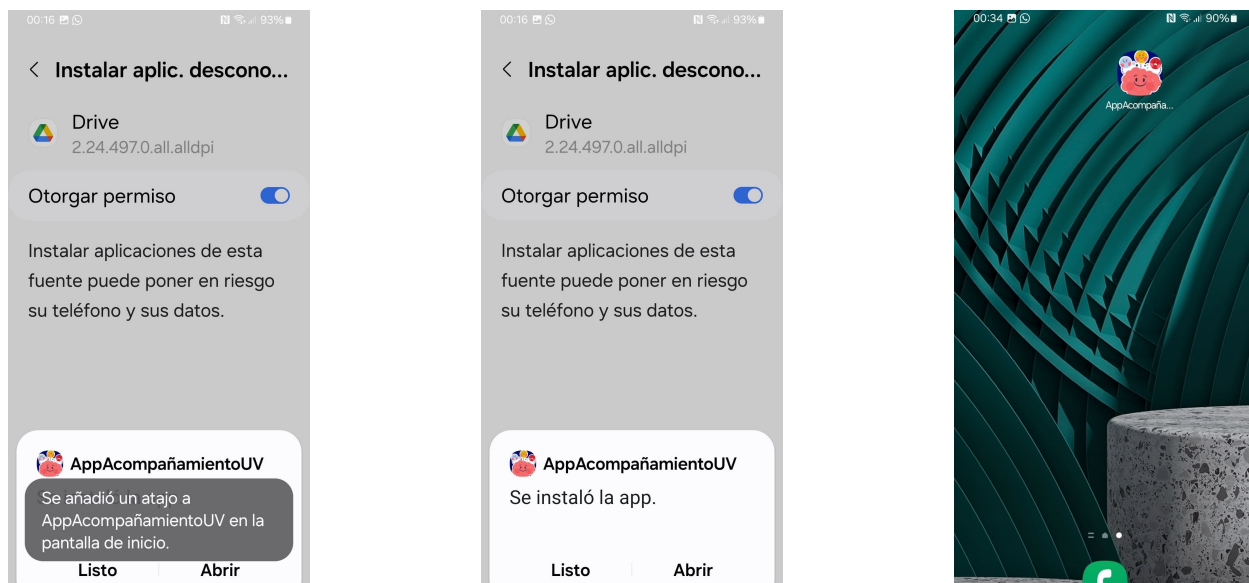


Figura 21: Instalación completada y acceso a App Acompañamiento UV.

Después de la instalación se debe comprobar la apertura de la aplicación, el inicio de sesión y la conexión con el backend. Si ya no se instalarán otros archivos desde la misma fuente, se recomienda revocar el permiso de instalación concedido temporalmente.

Nota adicional: Ante problemas durante la descarga o instalación, se puede solicitar apoyo al correo [sebastian.gonzalez@estudiantes.uv.cl](mailto:sebastian.gonzalez@estudiantes.uv.cl).

## **9. Criterios de continuidad**

Para que otra persona pueda mantener el sistema, el repositorio debe conservar actualizadas la estructura de carpetas, las instrucciones de ejecución, los perfiles de EAS y los nombres de las variables de entorno. Los valores sensibles deben entregarse por separado y limitarse a las personas autorizadas.

Cada versión distribuida debe asociarse a un estado identificable del repositorio. Cuando cambien las dependencias, el proveedor de alojamiento, las variables o los comandos de construcción, esta guía debe actualizarse junto con el código.

El entorno descrito permite reproducir el desarrollo y el despliegue utilizado en el proyecto. Antes de una operación institucional sostenida, el equipo o estudiante que continúe el trabajo deberá coordinar con la Dirección de Asuntos Estudiantiles y, cuando corresponda, con el Departamento de Tecnologías de Información y Comunicación las condiciones técnicas, responsabilidades y recursos necesarios.